

Now I have all the information. Let me create a comprehensive guide for you.

FluentValidation - Complete Guide Using Your Project

Part 1: Do You Use FluentValidation?

Answer: YES! You ARE using FluentValidation in your project.

Location: Only in **Ordering Service** (not in other services)

Current Usage:

-  CreateOrderCommandValidator - Validates order creation
-  UpdateOrderCommandValidator - Validates order updates
-  Integrated with CQRS pattern using decorators
-  Automatic validation before command handlers execute

Part 2: Why FluentValidation is Used

The Problem Without FluentValidation

Without FluentValidation (Manual Validation):

```
public async Task<int> Handle(CreateOrderCommand command,
CancellationTokentoken cancellationToken)
{
    // Validation scattered everywhere
    if (string.IsNullOrEmpty(command.UserName))
        throw new ArgumentException("UserName is required");

    if (command.UserName.Length > 70)
        throw new ArgumentException("UserName must not exceed 70
chars");

    if (command.TotalPrice < 0)
```

```

        throw new ArgumentException("TotalPrice shouldn't be
negative");

        if (!IsValidEmail(command.EmailAddress))
            throw new ArgumentException("Invalid email address");

        if (!string.IsNullOrEmpty(command.CardNumber)
&& !IsValidCreditCard(command.CardNumber))
            throw new ArgumentException("Invalid credit card");

        // ... 20 more validation lines

        // Finally, business logic
        var order = new Order(command);
        await _orderRepository.AddAsync(order);
        return order.Id;
    }

```

Problems:

- ✗ Validation mixed with business logic
- ✗ Duplicated validation in multiple handlers
- ✗ Hard to maintain (change validation in 10 places)
- ✗ No consistent error messages
- ✗ Violates Single Responsibility Principle

The Solution With FluentValidation

With FluentValidation (Clean Separation):

```

// Validator in separate class
public class CreateOrderCommandValidator :
AbstractValidator<CreateOrderCommand>
{
    public CreateOrderCommandValidator()
    {
        RuleFor(o => o.UserName).NotEmpty().MaximumLength(70);
        RuleFor(o => o.TotalPrice).GreaterThanOrEqualTo(0);
    }
}

```

```

        RuleFor(o => o.EmailAddress).NotEmpty().EmailAddress();
        // ... all validation rules in one place
    }
}

// Handler is now clean
public async Task<int> Handle(CreateOrderCommand command,
CancellationTokens cancellationTokens)
{
    // No validation here - automatically done by decorator!
    var order = new Order(command);
    await _orderRepository.AddAsync(order);
    return order.Id;
}

```

Benefits:

-  Validation separated from business logic
-  Reusable across multiple handlers
-  Consistent error messages
-  Easy to test
-  Follows Single Responsibility Principle
-  Built-in validators (Email, CreditCard, Regex, etc.)

Part 3: How You Implemented FluentValidation in Ordering Service

Step 1: Install NuGet Packages

File: Ordering.Application.csproj (Lines 17-18)

```

<PackageReference Include="FluentValidation" Version="12.1.1" />
<PackageReference
Include="FluentValidation.DependencyInjectionExtensions"
Version="12.1.1" />

```

Step 2: Create Validator Classes

File: Ordering.Application/Validators/CreateOrderCommandValidator.cs

```
using FluentValidation;
using Ordering.Application.Orders.CreateOrder;

namespace Ordering.Application.Validators
{
    public class CreateOrderCommandValidator :
    AbstractValidator<CreateOrderCommand>
    {
        public CreateOrderCommandValidator()
        {
            RuleFor(o => o.UserName)
                .NotEmpty().WithMessage("{PropertyName} is Required.")
                .MaxLength(70).WithMessage("{PropertyName} must
not exceed 70 chars");

            RuleFor(o => o.TotalPrice)
                .NotEmpty().WithMessage("{PropertyName} is
required.")
                .GreaterThanOrEqualTo(0).WithMessage("{PropertyName}
shouldn't be negative");

            RuleFor(o => o.EmailAddress)
                .NotEmpty().WithMessage("{PropertyName} is
required.")
                .EmailAddress().WithMessage("{PropertyName} must be a
valid email address.");

            RuleFor(o => o.FirstName)
                .NotEmpty().WithMessage("{PropertyName} is required.");

            RuleFor(o => o.LastName)
                .NotEmpty().WithMessage("{PropertyName} is
required.");

            RuleFor(o => o.CardNumber)
```

```

        .CreditCard().When(o
=> !string.IsNullOrEmpty(o.CardNumber))
        .WithMessage("{PropertyName} must be a valid credit
card");

    RuleFor(o => o.Expiration)
        .Matches(@"^(0[1-9]|1[0-2])\./?([0-9]{2})$") // MM/YY
format
        .When(o => !string.IsNullOrEmpty(o.Expiration))
        .WithMessage("{PropertyName} must be in MM/YY
format.");

    RuleFor(o => o.Cvv)
        .Matches(@"^\d{3,4}$")
        .When(o => !string.IsNullOrEmpty(o.Cvv))
        .WithMessage("{PropertyName} must be 3 or 4 digits.");
    }
}
}

```

File: Ordering.Application/Validators/UpdateOrderCommandValidator.cs

```

using FluentValidation;
using Ordering.Application.Orders.UpdateOrder;

namespace Ordering.Application.Validators
{
    public class UpdateOrderCommandValidator :
AbstractValidator<UpdateOrderCommand>
    {
        public UpdateOrderCommandValidator()
        {
            RuleFor(o => o.Id)
                .NotEmpty().WithMessage("{PropertyName} is required")
                .GreaterThan(0).WithMessage("{PropertyName} must be
greater than 0.");

            RuleFor(o => o.UserName)
                .NotEmpty().WithMessage("{PropertyName} is required.")

```

```
        .MaxLength(70).WithMessage("{PropertyName} must not  
exceed 70 chars");
```

```
        RuleFor(o => o.TotalPrice)  
            .NotEmpty().WithMessage("{PropertyName} is required.")  
            .GreaterThanEqualTo(0).WithMessage("{PropertyName}  
shouldn't be negative");
```

```
        RuleFor(o => o.EmailAddress)  
            .NotEmpty().WithMessage("{PropertyName} is required")  
            .EmailAddress().WithMessage("{PropertyName} must be a  
valid email address.");
```

```
        RuleFor(o => o.FirstName)  
            .NotEmpty().WithMessage("{PropertyName} is required.");
```

```
        RuleFor(o => o.LastName)  
            .NotEmpty().WithMessage("{PropertyName} is  
required.");
```

```
        RuleFor(o => o.CardNumber)  
            .CreditCard().When(o  
=> !string.IsNullOrEmpty(o.CardNumber))  
            .WithMessage("{PropertyName} must be a valid credit  
card number.");
```

```
        RuleFor(o => o.Expiration)  
            .Matches(@"^(0[1-9]|1[0-2])\/?([0-9]{2})$") // MM/YY  
format  
            .When(o => !string.IsNullOrEmpty(o.Expiration))  
            .WithMessage("{PropertyName} must be in MM/YY  
format.");
```

```
        RuleFor(o => o.Cvv)  
            .Matches(@"^\d{3,4}$")  
            .When(o => !string.IsNullOrEmpty(o.Cvv))  
            .WithMessage("{PropertyName} must be 3 or 4 digits.");  
    }  
}
```

```
}
```

Step 3: Create Validation Decorator (CQRS Integration)

File: Ordering.Application/Behaviours/ValidationCommandHandlerDecorator.cs

```
using FluentValidation;
using Ordering.Application.Abstractions;
using System.ComponentModel.DataAnnotations;
using ValidationException = FluentValidation.ValidationException;

namespace Ordering.Application.Behaviours
{
    // HTTP Request → Command → [Exception Decorator] → [Validation
    // Decorator] → Real Handler → Response
    // ✅ WITH decorator – Fluent validation is CENTRALIZED here,
    // handlers stay clean
    public class ValidationCommandHandlerDecorator<TCommand,TResult> :
    ICommandHandler<TCommand,TResult>
        where TCommand : ICommand<TResult>
    {
        private readonly ICommandHandler<TCommand, TResult> _inner;
        private readonly IEnumerable<IValidator<TCommand>>
        _validators;

        public
        ValidationCommandHandlerDecorator(ICommandHandler<TCommand, TResult>
        inner,
            IEnumerable<IValidator<TCommand>> validators)
        {
            _inner = inner;
            _validators = validators;
        }

        public async Task<TResult> Handle(TCommand command,
        CancellationToken cancellationToken)
        {
            if(_validators.Any())
```

```

        {
            var context = new
ValidationContext<TCommand>(command);

            // Run ALL validators for this command in parallel
            var results = await
Task.WhenAll(_validators.Select(v=>v.ValidateAsync(context,cancellation
Token)));

            // Collect all failures from all validators
            var failures = results
                .SelectMany(r=>r.Errors)
                .Where(f=>f !=null)
                .ToList();

            if(failures.Any())
            {
                throw new ValidationException(failures); // inner
handler NOT called
            }
        }
        return await _inner.Handle(command, cancellationToken); //
✓ only runs if VALID
    }
}
}

```

What this decorator does:

1. Intercepts every command before it reaches the handler
2. Finds all validators for that command type
3. Runs all validators in parallel
4. Collects all validation errors
5. Throws exception if validation fails (handler never runs)
6. Only calls handler if validation passes

Step 4: Register in DI Container

File: Ordering.API/Extensions/ServiceCollectionExtensions.cs (Lines 59-64)

```
//4 Fluent Validation
services.AddValidatorsFromAssembly(typeof(CreateOrderCommandValidator)
.Assembly);

// 5. Decorators Pipeline
services.Decorate(typeof(ICommandHandler<, >),
typeof(ValidationCommandHandlerDecorator<, >));
services.Decorate(typeof(ICommandHandler<, >), typeof(UnhandledException
CommandHandlerDecorator<, >));
```

Breakdown:

- AddValidatorsFromAssembly: Scans assembly and registers all validators automatically
- Decorate: Wraps every command handler with validation decorator
- Order matters: Validation runs before exception handling

Part 4: How It Works - Complete Flow

Request Flow with Validation

HTTP Request (POST /api/orders)
↓
Controller receives CreateOrderCommand
↓
Controller calls: await _mediator.Send(command)
↓
MediatR finds handler
↓
Validation Decorator intercepts
↓

Validation Decorator Logic: 1. Find all validators for command

- | |
|---|
| <ol style="list-style-type: none">2. Run validators in parallel3. Collect all errors4. If errors → throw exception5. If valid → call inner handler |
|---|



If validation passes → Real Handler executes



Handler saves order to database



Returns order ID

Validation Rules in Your Project

CreateOrderCommandValidator validates:

-  UserName: Required, max 70 chars
-  TotalPrice: Required, not negative
-  EmailAddress: Required, valid email format
-  FirstName: Required
-  LastName: Required
-  CardNumber: Valid credit card (if provided)
-  Expiration: MM/YY format (if provided)
-  CVV: 3-4 digits (if provided)

Part 5: Comparison - With vs Without FluentValidation

Without FluentValidation (Bad Practice)

```
public class CreateOrderHandler : ICommandHandler<CreateOrderCommand,
int>
{
    public async Task<int> Handle(CreateOrderCommand command,
CancellationTokentoken cancellationToken)
    {
        //  Validation mixed with business logic
        if (string.IsNullOrEmpty(command.UserName))
```

```

        throw new ArgumentException("UserName is required");

    if (command.UserName.Length > 70)
        throw new ArgumentException("UserName too long");

    if (command.TotalPrice < 0)
        throw new ArgumentException("Price cannot be negative");

    // ✗ 15 more validation lines...

    // Finally business logic
    var order = new Order(command);
    await _orderRepository.AddAsync(order);
    return order.Id;
}
}

// ✗ Same validation copied to UpdateOrderHandler
// ✗ Same validation copied to DeleteOrderHandler
// ✗ Hard to maintain, easy to forget

```

With FluentValidation (Good Practice - Your Implementation)

```

// ✔ Validator in one place
public class CreateOrderCommandValidator :
    AbstractValidator<CreateOrderCommand>
{
    public CreateOrderCommandValidator()
    {
        RuleFor(o => o.UserName).NotEmpty().MaximumLength(70);
        RuleFor(o => o.TotalPrice).GreaterThanOrEqualTo(0);
        // ... all rules in one place
    }
}

// ✔ Handler is clean
public class CreateOrderHandler : ICommandHandler<CreateOrderCommand,

```

```

int>
{
    public async Task<int> Handle(CreateOrderCommand command,
CancellationTokentoken cancellationToken)
    {
        //  No validation here - automatically done!
        var order = new Order(command);
        await _orderRepository.AddAsync(order);
        return order.Id;
    }
}

```

Part 6: Step-by-Step Guide to Add FluentValidation to Other Services

Add to Basket Service

Step 1: Install Package

```

cd Services/Basket/Basket.Application
dotnet add package FluentValidation
dotnet add package FluentValidation.DependencyInjectionExtensions

```

Step 2: Create Validator

File: Basket.Application/Validators/CreateShoppingCartCommandValidator.cs

```

using FluentValidation;
using Basket.Application.Commands;

namespace Basket.Application.Validators
{
    public class CreateShoppingCartCommandValidator :
AbstractValidator<CreateShoppingCartCommand>
    {
        public CreateShoppingCartCommandValidator()

```

```

    {
        RuleFor(x => x.UserName)
            .NotEmpty().WithMessage("UserName is required")
            .MaxLength(50).WithMessage("UserName must not
exceed 50 characters");

        RuleFor(x => x.items)
            .NotNull().WithMessage("Items cannot be null")
            .Must(items => items.Count > 0).WithMessage("At least
one item is required");

        RuleForEach(x => x.items).ChildRules(item =>
        {
            item.RuleFor(i => i.ProductName)
                .NotEmpty().WithMessage("Product name is
required");

            item.RuleFor(i => i.Quantity)
                .GreaterThan(0).WithMessage("Quantity must be
greater than 0");

            item.RuleFor(i => i.Price)
                .GreaterThanOrEqualTo(0).WithMessage("Price cannot
be negative");
        });
    }
}

```

Step 3: Register in Program.cs

File: Basket.API/Program.cs (add after MediatR registration)

```

// Add FluentValidation
builder.Services.AddValidatorsFromAssembly(typeof(CreateShoppingCartCo
mmandValidator).Assembly);

// If using CQRS with decorators

```

```
builder.Services.Decorate(typeof(IRequestHandler<,>),  
typeof(ValidationBehavior<,>));
```

Step 4: Create Validation Behavior (for MediatR)

File: Basket.Application/Behaviors/ValidationBehavior.cs

```
using FluentValidation;  
using MediatR;  
  
namespace Basket.Application.Behaviors  
{  
    public class ValidationBehavior<TRequest, TResponse> :  
        IPipelineBehavior<TRequest, TResponse>  
        where TRequest : IRequest<TResponse>  
    {  
        private readonly IEnumerable<IValidator<TRequest>>  
_validators;  
  
        public ValidationBehavior(IEnumerable<IValidator<TRequest>>  
validators)  
        {  
            _validators = validators;  
        }  
  
        public async Task<TResponse> Handle(TRequest request,  
RequestHandlerDelegate<TResponse> next, CancellationToken  
cancellationToken)  
        {  
            if (_validators.Any())  
            {  
                var context = new  
ValidationContext<TRequest>(request);  
                var validationResults = await Task.WhenAll(  
                    _validators.Select(v => v.ValidateAsync(context,  
cancellationToken)));  
  
                var failures = validationResults
```

```

        .SelectMany(r => r.Errors)
        .Where(f => f != null)
        .ToList();

    if (failures.Any())
        throw new ValidationException(failures);
    }
    return await next();
}
}
}
}

```

Part 7: Built-in Validators Available

Validator	Description	Example
NotEmpty()	Must not be null or empty	RuleFor(x => x.Name).NotEmpty()
NotNull()	Must not be null	RuleFor(x => x.Items).NotNull()
MaxLength(n)	String length limit	RuleFor(x => x.Name).MaxLength(50)
MinLength(n)	Minimum string length	RuleFor(x => x.Password).MinLength(8)
GreaterThan(n)	Greater than value	RuleFor(x => x.Price).GreaterThan(0)
GreaterThanOrEqualTo(n)	Greater or equal	RuleFor(x => x.Quantity).GreaterThanOrEqualTo(0)

LessThan(n)	Less than value	RuleFor(x => x.Age).LessThan(120)
EmailAddress()	Valid email format	RuleFor(x => x.Email).EmailAddress()
CreditCard()	Valid credit card	RuleFor(x => x.Card).CreditCard()
Matches(regex)	Regex pattern	RuleFor(x => x.Phone).Matches(@"^\d{10}\$")
Must(predicate)	Custom condition	RuleFor(x => x.Age).Must(BeAdult)
When(condition)	Conditional rule	.When(x => x.HasCard)

Part 8: Your Current Status

Service	FluentValidation Status
Ordering	✅ Implemented with validators and decorators
Basket	❌ Not implemented (can add)
Catalog	❌ Not implemented (can add)
Discount	❌ Not implemented (can add)
Payment	❌ Not implemented (can add)

Summary

You ARE using FluentValidation in Ordering service with:

- ✅ CreateOrderCommandValidator
- ✅ UpdateOrderCommandValidator

-  Validation decorator for automatic validation
-  Clean separation of validation from business logic
-  Integration with CQRS pattern

Benefits you're getting:

- Clean handlers without validation clutter
- Reusable validation rules
- Consistent error messages
- Easy to test
- Follows SOLID principles

Recommendation: Add FluentValidation to other services (Basket, Catalog) following the same pattern used in Ordering service.